School of Future Tech

CASE STUDY REPORT ON

Electricity Bill Generator

by

Ramjee Mishra

150096725007

1. Introduction to the Case Study

In modern utility management systems, accuracy and efficiency are paramount. Electricity billing, a fundamental service for households and industries, involves complex calculations based on variable consumption slabs, fixed operational charges, and government taxes. Manual calculation of these bills is prone to human error, time-consuming, and inefficient for large customer bases.

This case study explores the development of an automated "Electricity Bill Generator" using the C++ programming language. The system is designed to streamline the billing process by accepting customer usage data and applying specific tariff rules to generate a detailed, accurate bill. By automating this process, utility providers can ensure transparency, reduce administrative overhead, and provide customers with clear breakdowns of their energy costs.

2. Case Study Background

Background

Traditional electricity billing often relied on manual meter reading and calculation. This approach faces several challenges, including calculation errors due to complex slab-based pricing structures, delays in bill generation, and a lack of detailed consumption analysis for the consumer. As energy consumption grows, the need for a robust, digital solution becomes critical.

Abstract

The problem addressed in this case study is the accurate computation of electricity charges based on a tiered (slab-based) tariff structure. The project implements a C++ console application that takes customer details and units consumed as input. It dynamically calculates the bill amount by applying different rates to different chunks of energy consumption, adds a mandatory fixed charge, and computes the Goods and Services Tax (GST). The solution utilizes modular programming concepts, data structures (structs), and input validation to create a reliable billing tool.

3. Case Study Design

The Electricity Bill Generator is designed as a modular pipeline to ensure code maintainability and clarity. The system architecture consists of the following components:

1. **Data Model:** A structure to hold customer information (Name, ID, Units).
2. **Input Module:** A robust interface for capturing user data with validation to prevent negative values or invalid types.
3. **Calculation Engine:** The core logic that applies the slab rates. This separates the business logic from the presentation layer.
4. **Taxation Module:** A dedicated function to compute applicable taxes on the base bill amount.
5. **Output/Reporting Module:** Formats the final results into a readable bill, displaying the breakdown of charges.

4. Methods & Algorithms Technology Applied

The core of the system relies on a slab-based tariff algorithm. The implementation details are as follows:

Tariff Structure

Consumption Slab (Units)	Rate per Unit (Rs)
0 - 100	1.50
101 - 300	2.50
301 - 500	4.00
501 and above	6.00

Additional Charges

- **Fixed Charge:** Rs 50.00 (Applied to every bill regardless of consumption).
- **Tax (GST):** 18% calculated on the total of energy charges and fixed charges.

Technology Stack

- **Language:** C++.
- **Libraries:**
  - iostream: For input/output operations.
  - iomanip: For formatting currency and table alignment.
  - limits: For handling input buffer clearing during validation.
  - string: For string manipulation.

5. Implementation Details and Snapshots

Key Code Modules

The implementation is divided into distinct functions to handle specific tasks:

1. Data Structure

A Struct is used to group customer-related data, keeping the code organized.

2. Bill Calculation Logic

The calculateBill function implements the "if-else if" ladder to calculate costs cumulatively across slabs. This ensures that a customer consuming 350 units pays the lower rate for the first 100 units, the medium rate for the next 200, and the higher rate only for the remaining 50.

3. Input Validation

The getValidInt function ensures robust error handling. It loops until the user provides a non-negative integer, protecting the system from crashing due to invalid characters.

Sample Output Scenario

Below is a representation of the console output generated by the system for a sample customer.

```
-----------------------------------------
Customer Name        : Ramjee
Customer ID          : 10052 Units
Consumed             : 350 kWh
-----------------------------------------
Slab Rates Applied:
                 0  - 100  units @ Rs 1.50/unit
               101 - 300  units @ Rs 2.50/unit
               301 - 500  units @ Rs 4.00/unit
               500+       units @ Rs 6.00/unit
-----------------------------------------
Fixed Charge : Rs 50.00 Base Amount:
Rs 900.00 Bill Amount   : Rs 950.00 Tax
(18.00% GST): Rs 171.00
-----------------------------------------
Total Payable: Rs 1121.00
-----------------------------------------
```

6. Results and Conclusion

Findings

The implemented C++ program successfully meets the requirements of a modern billing system. Key findings include:

- **Accuracy:** The slab-based logic correctly differentiates between consumption tiers, ensuring fair pricing.
- **Robustness:** The input validation mechanisms effectively prevent runtime errors caused by incorrect user input (e.g., entering text instead of numbers).
- **Clarity:** The formatted output using iomanip provides a professional bill layout that is easy to read and verify.

Conclusion

This case study demonstrates that a modular C++ application can effectively solve real-world administrative problems like utility billing. By automating the calculation of variable rates and taxes, the system minimizes human error and significantly reduces the time required to process bills. The resulting software is a scalable foundation that could be expanded with features like file storage, historical data analysis, or a graphical user interface.

7. References

- C++ Reference Documentation. *Input/Output Manipulators*. Retrieved from https://en.cppreference.com/w/cpp/io/manip
- GeeksforGeeks. *Structures in C++*. Retrieved from https://www.geeksforgeeks.org/structures-in-cpp/
- Standard Tariff Regulations. *Electricity Duty and Tax Structure*.